

LAB 2

1. ArrayList – add and access elements

```
using System.Collections;

class Program
{
    static void Main(string[] args)
    {
        ArrayList items = new ArrayList();
        items.Add("Mango");
        items.Add('A');
        items.Add(150);
        items.Add(9.8);
        items.Add(true);
        Console.Write("Items: ");
        foreach(var item in items)
            Console.Write(item + " ");
        Console.WriteLine();
        Console.WriteLine("Item at index 1 : " + items[0]);
        Console.ReadLine();
    }
}
```

```
yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Items: Mango A 150 9.8 True
Item at index 1 : Mango
```

2. Hashtable – add and access key – value pairs

```
using System.Collections;

class Program
{
    static void Main(string[] args)
    {
        Hashtable hashtable = new Hashtable();
        hashtable.Add("S101", "Ram");
        hashtable.Add("S102", "Raj");
        hashtable.Add("S103", "Ravi");
        Console.WriteLine("Hashtable elements : ");
        foreach(DictionaryEntry de in hashtable)
            Console.WriteLine(de.Key + " " + de.Value);
        Console.WriteLine("Access value for key 'S101' : "+hashtable["S101"]);
        Console.ReadLine();
    }
}
```

```

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Hashtable elements :
S103 Ravi
S101 Ram
S102 Raj
Access value for key 'S101' : Ram

```

3. Non - generic queue – enqueue , dequeue, traverse using System.Collections;

```

class Program
{
    static void Main(string[] args)
    {
        Queue q = new Queue();
        q.Enqueue('A');
        q.Enqueue("Ram");
        q.Enqueue(true);
        q.Enqueue(1.23);
        Console.WriteLine("Top of the queue : " + q.Peek());
        Console.Write("Elements in the queue : ");
        foreach(var item in q)
            Console.Write(item + " ");
        Console.WriteLine();
        Console.Write("Removing elements from the queue : ");
        while(q.Count > 0)
            Console.Write(q.Dequeue() + " ");
        Console.WriteLine();
    }
}

```

```

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Top of the queue : A
Elements in the queue : A Ram True 1.23
Removing elements from the queue : A Ram True 1.23

```

4. Non – generic stack – push, pop, traverse using System.Collections;

```

class Program
{
    static void Main(string[] args)
    {
        Stack s = new Stack();
        s.Push(10);
        s.Push('A');
        s.Push(20.5);
        s.Push(true);
        Console.WriteLine("Top of the stack : " + s.Peek() );
        Console.Write("Elements in the stack : ");
        foreach(var item in s)

```

```

        Console.Write(item + " ");
        Console.WriteLine();
        Console.Write("Removing elements from the queue : ");
        while(s.Count > 0)
            Console.Write(s.Pop() + " ");
        Console.WriteLine();
    }
}

```

```

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Top of the stack : True
Elements in the stack : True 20.5 A 10
Removing elements from the queue : True 20.5 A 10

```

5. List<int> - add, access, iterate

using System.Collections;

```

class Program
{
    static void Main(string[] args)
    {
        List<int> numbers = new List<int>();
        numbers.Add(23);
        numbers.Add(42);
        numbers.Add(12);
        Console.WriteLine("Elements of the list: ");
        foreach (int number in numbers)
        {
            Console.Write(number + " ");
        }
        Console.WriteLine();
        Console.WriteLine("Element at index 1: " + numbers[1]);
    }
}

```

```

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Elements of the list:
23 42 12
Element at index 1: 42

```

6. Dictionary<String, int> - add, access, iterate

using System.Collections;

```

class Program
{
    static void Main(string[] args)
    {
        Dictionary<string, int> students = new Dictionary<string, int>();
        students.Add("John", 20);
        students.Add("Jane", 22);
        students.Add("Bob", 19);
        Console.WriteLine("Elements in the dictionary: ");
        foreach (var student in students)

```

```

    {
        Console.WriteLine("Name : " + student.Key + " age : " + student.Value);
    }
    Console.WriteLine("Age of Jane : " + students["Jane"]);
}
}
yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Elements in the dictionary:
Name : John age : 20
Name : Jane age : 22
Name : Bob age : 19
Age of Jane : 22

```

7. Queue<string> - generic enqueue, dequeue, traverse

using System.Collections;

```

class Program
{
    static void Main(string[] args)
    {
        Queue<string> cities = new Queue<string>();
        cities.Enqueue("New York");
        cities.Enqueue("London");
        cities.Enqueue("Paris");
        cities.Enqueue("Tokyo");
        Console.WriteLine("Cities in the queue: ");
        foreach (string city in cities)
            Console.Write(city + " ");
        Console.WriteLine();
        Console.WriteLine("Extracting elements: ");
        while (cities.Count > 0)
            Console.Write(cities.Dequeue() + " ");
        Console.WriteLine();
    }
}
yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Cities in the queue:
New York London Paris Tokyo
Extracting elements:
New York London Paris Tokyo

```

8. Stack<string> - generic push, pop, traverse

using System.Collections;

```

class Program
{
    static void Main(string[] args)
    {
        Stack<string> cities = new Stack<string>();
        cities.Push("New York");
    }
}

```

```

        cities.Push("London");
        Console.WriteLine("Cities in the queue: ");
        foreach (string city in cities)
            Console.Write(city + " ");
        Console.WriteLine();
        Console.WriteLine("Extracting elements: ");
        while (cities.Count > 0)
            Console.Write(cities.Pop() + " ");
        Console.WriteLine();
    }
}

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Cities in the queue:
London  New York
Extracting elements:
London  New York

```

9. Filter even numbers

using System.Collections;

```

class Program
{
    static void Main(string[] args)
    {
        List<int> numbers = new List<int> { 23, 18, 25, 10 };
        var even = from num in numbers
                    where num % 2 == 0
                    select num;
        Console.WriteLine("Even numbers: ");
        foreach (var number in even)
        {
            Console.Write(number + "\t");
        }
        Console.WriteLine();
    }
}

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Even numbers: 18      10

```

10. Filter and sort integers ascending

using System.Collections;

```

class Program
{
    static void Main(string[] args)
    {
        List<int> numbers = new List<int> { 23, 18, 25, 10, 22, 13 };
        var number = from num in numbers
                      where num > 15
                      orderby num ascending
                      select num;
    }
}

```

```

        Console.WriteLine("Numbers greater than 15 in ascending order: ");
        foreach (var a in number)
        {
            Console.WriteLine(a+"\t");
        }
        Console.WriteLine();
    }
}

```

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2\$ dotnet run

Numbers greater than 15 in ascending order: 18 22 23 25

11. Sorts strings alphabetically

```
using System.Collections;
```

```

class Program
{
    static void Main(string[] args)
    {
        List<string> names = new List<string> { "Ram", "Harry", "Tom" };
        var name = from n in names
                    orderby n
                    select n;
        Console.WriteLine("Names in ascending order: ");
        foreach (var a in name)
        {
            Console.WriteLine(a+"\t");
        }
        Console.WriteLine();
    }
}

```

```

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Names in ascending order: Harry Ram        Tom

```

12. Sort integers descending

```
using System.Collections;
```

```

class Program
{
    static void Main(string[] args)
    {
        List<int> numbers = new List<int> { 23, 45, 21, 34 };
        var sortedNumbers = from number in numbers
                             orderby number descending
                             select number;
        Console.WriteLine("Sorted numbers: ");
        foreach (var number in sortedNumbers)
        {
            Console.WriteLine(number + "\t");
        }
        Console.WriteLine();
    }
}

```

```

    }
}

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Sorted numbers:
45      34      23      21

```

13. Filter strings by prefix and sort

using System.Collections;

```

class Program
{
    static void Main(string[] args)
    {
        List<string> names = new List<string>{"Tom", "Jerry", "Oliver", "John", "Jack"};
        var name = from item in names
                    where item.StartsWith('J')
                    orderby item
                    select item;
        foreach (var item in name)
        {
            Console.Write(item + "\t");
        }
        Console.WriteLine();
    }
}

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Jack    Jerry   John

```

14. Multi – level sort on primitives

using System.Collections;

```

class Program
{
    static void Main(string[] args)
    {
        List<string> names = new List<string>{"Tom", "Jerry", "Oliver", "Janney", "Jackson",
        "Harry", "Jack"};
        var name = from item in names
                    orderby item, item.Length
                    select item;
        foreach (var item in name)
        {
            Console.Write(item + "\t");
        }
        Console.WriteLine();
    }
}

yanil@fedora:~/Desktop/6th_sem/DotNet/lab/Lab2$ dotnet run
Harry   Jack    Jackson Janney   Jerry   Oliver   Tom

```